Seminar in Software Engineering

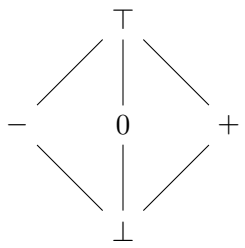
HW1 Workshop – Zero Analysis

READ THE ENTIRE ASSIGNMENT BEFORE STARTING.

Deadline: October 27, 2026 **(15:59) (no extensions)**

Introduction

A `zero-analysis` is a *may forward dataflow analysis* whose goal is to detect whether an integer variable may be zero (or not) during the execution of a program, and so whether a division may divide by zero. The analysis represents the state as a function from program variables to a lattice of abstract values $Sign = \{\bot, -, 0, +, \top\}$:



Therefore, the value $IN[n]$ of a node n of the control-flow graph is an abstract state that we can model as follows:

- $IN[n](x) = \bot$ (bottom) if x never had a value on entry to the node.
- $IN[n](x) = 0$ (zero) if the value of x is always zero on entry to the node.
- $IN[n](x) = +$ (positive) if the value of x is always positive on entry to the node.
- $IN[n](x) = -$ (negative) if the value of x is always negative on entry to the node.
- $IN[n](x) = \top$ (top) if x may have any value on entry to the node.

In the same way, we can interpret the possible values of x for the abstract state $OUT[n]$.
The dataflow equations that characterize this analysis are the following:

$$IN[n] = \bigsqcup_{n' \in pred(n)} OUT[n']$$

$$OUT[n] = Transfer[n, IN[n]]$$

A division `x = y / z` is reported as a possible division by zero when $IN[n](z)$ may be zero.

Part 1: Defining the Zero Analysis

The exercises in Part 1 are a warm-up before starting the implementation. They are not submitted, but they serve as a basis for discussing any questions about the semantics of the analysis.

Let $x, y, z \in \mathbb{Z}$**.** Ignore overflow: the operations are those of the integers.

Exercise 1

Method ZeroValueVisitor::visitIntegerConstant

Complete the following table with the expected values of the abstract state $OUT[n]$ for the variable x when n is the assignment of a constant, with $K \in \mathbb{Z}$:

n	OUT[n](x)
`x = 0`	
`x = K` // with $K > 0$	
`x = K` // with $K < 0$	

Exercise 2

Method ZeroValueVisitor::visitLocal

Complete the following table to characterize the transfer function ($Transfer$) for the copy statement (`x = y`):

IN[n](y)	OUT[n](x)
$\bot$	
...	

Exercise 3

Methods ZeroAbstractValue::add, substract, multiplyBy and divideBy

Complete the following tables to characterize the transfer function ($Transfer$) of the statements `x = y + z`, `x = y - z`, `x = y * z` and `x = y / z` (an **integer** division). Each row is the value of $IN[n](y)$ and each column the value of $IN[n](z)$; each cell is the value of $OUT[n](x)$.

$y + z$	$\bot$	$-$	0	$+$	$\top$
$\bot$					
$-$					
0					
$+$					
$\top$					

$y - z$	$\bot$	$-$	0	$+$	$\top$
$\bot$					
$-$					
0					
$+$					
$\top$					

$y \times z$	$\bot$	$-$	0	$+$	$\top$
$\bot$					
$-$					
0					
$+$					
$\top$					

y/z	$\bot$	$-$	0	$+$	$\top$
$\bot$					
$-$					
0					
$+$					
$\top$					

Hint: A division by zero has no result. Remember that integer division truncates: $1/2 = 0$.

Exercise 4

Method ZeroAbstractValue::merge

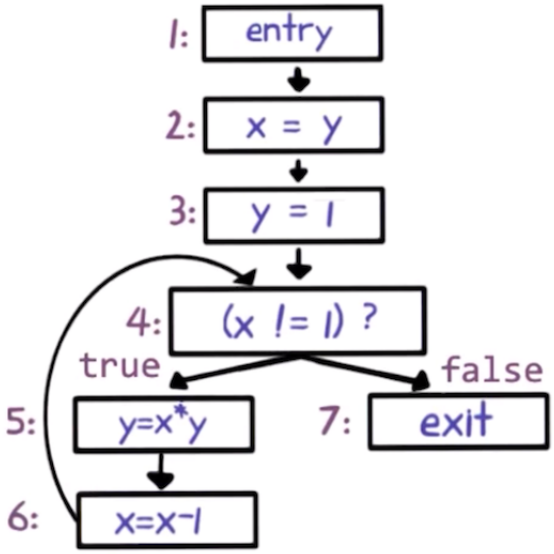
Complete the table of $a \sqcup b$, the least upper bound of two abstract values in the lattice, used to merge the states that reach a node from its predecessors.

$a \sqcup b$	$\bot$	$-$	0	$+$	$\top$
$\bot$					
$-$					
0					
$+$					
$\top$					

Exercise 5

Consider the following program and its corresponding control-flow graph:

```
public int factorial(int y) {
  int x = y;
  y = 1;
  while (x != 1) {
    y = x * y;
    x = x - 1;
  }
  return y;
}
```



Run the chaotic iteration algorithm until stable values of IN and OUT are obtained for every variable of the program. **Note that the initial value of the parameters is always $\top$, since we do not know what value they may take when the program is executed.**

n	IN[n](x)	IN[n](y)	OUT[n](x)	OUT[n](y)
1	$\bot$	$\top$		
2				
3				
4				
5				
6				
7				

Part 2: Implementing the Zero Analysis with Tai-e

For the workshop to work, a Java Development Kit (JDK) 17 or newer must be installed. Gradle and Tai-e are downloaded automatically the first time you build the project (see `STUDENTS.md` in the repo).

Tai-e

Tai-e is a static analysis framework for Java. It translates the program's bytecode into an intermediate representation (IR), builds control-flow graphs, and provides a framework for *dataflow analyses*, including a worklist solver that computes the fixpoint. You can find more information at the following links:

- Project page: `https://github.com/pascal-lab/Tai-e`
- Reference documentation:

  `https://tai-e.pascal-lab.net/docs/current/reference/en/index.html`
- Java API documentation:

  `https://tai-e.pascal-lab.net/docs/current/api/index.html`

Tai-e's intermediate representation

The analysis does not work on the Java source, but on Tai-e's IR: a typed, 3-address representation in which each statement (`Stmt`) does at most one operation. An assignment `x = e` is a `DefinitionStmt`, whose right-hand side is one of the expressions the visitor handles: an integer constant (`IntLiteral`), a variable (`Var`), or an arithmetic operation between two variables (`ArithmeticExp`). The IR stays close to the source, but not identical. In particular:

- Constants are never operands: `x = y + 1` becomes `%ci1 = 1; x = y + %ci1`. Variables whose names start with `%` are temporaries created by Tai-e.
- A variable that is reassigned with an independent value may be split into several variables: `x` and `x#1`.
- A variable that always holds the same constant is assigned once, at the start of the method; statements with no source line report line `-1`.
- Method parameters are not defined by any statement: their value at the start of the method is computed by the visitor's `visitParameter` method.

The command `./gradlew zeroAnalysis -PtargetClass=<targetClass>` prints the IR of each method of the class, together with the IN state your analysis computed for each statement. We recommend running it on the `targets` classes to get familiar with the IR.

Implementation

In this workshop you are asked to implement a dataflow analysis that infers information about the value of an integer variable in order to detect whether there is a division by zero. To this end, you will have to complete an existing implementation: the repo you get from the course.

The implementation must support (at least) the following operations:

- `x = 0;`
- `x = K;` // where `K` is a non-zero constant
- `x = y;`
- `x = y + z;`

- `x = y - z;`
- `x = y * z;`
- `x = y / z;` // integer division

You must complete the methods of the `enum ZeroAbstractValue`; the methods `maybeZeroOperand` and `visitIntegerConstant` of the class `ZeroValueVisitor`; the method `union` of the class `ZeroAbstractState`; and (we recommend using the previous method) the method `meetInto` of the class `DivisionByZeroAnalysis`, which merges the OUT state of a predecessor into the IN state of a statement.

Some of the files and folders in the project are:

- **targets** contains the classes to be analyzed in the workshop.
- **util** contains the base of the visitor design pattern for visiting the different **expressions** of the program (`util.visitor`), and the code that loads a class with Tai-e and builds the IR and control-flow graph of its methods (`util.taie`).
- **zeroanalysis** is the folder containing the code to be modified.
  - **DivisionByZeroAnalysis:** the class that implements the zero-analysis (extends Tai-e's `AbstractDataflowAnalysis`: it defines the initial states, the transfer function `transferNode` and the merge `meetInto`, and Tai-e's solver computes the fixpoint).
  - **ZeroValueVisitor:** contains the implementation of the visitor design pattern for visiting the different **expressions** of the program (extends `AbstractValueVisitor` from utils).
  - **ZeroAbstractState:** a mapping from variable names to a `ZeroAbstractValue`.
  - **ZeroAbstractValue:** implements the abstract values mentioned above.
- **Launcher** (package `seminar.dataflow`): the *entry point* of the analyzer.
- **DivisionByZeroTest** (in `src/test`): the tests that the implementation must pass.

You can run the project's tests from your IDE or from the terminal with the command `./gradlew test` (`gradlew.bat test` on Windows). If you want to run the Division By Zero Analysis on a class from the terminal, you can do so with the command `./gradlew zeroAnalysis -PtargetClass=<targetClass>`.

The implemented analysis must compute the dataflow information correctly. The tests included in the project read the Java programs included in the `targets` folder and check that the abstract values of the variables at certain program points are computed correctly and that a potential division by zero is detected. The course also grades your submission with additional tests, which check every operation of `ZeroAbstractValue` on every pair of values.

Submission

A complete implementation of the Zero Analysis. The tests provided by the teaching staff must pass. You may add your own if you wish.